Analyse des méthodes de transformation d'une maquette en modèle hiérarchique pour la génération de code frontend

Étude comparative : ScreenCoder, DesignCoder, Prototype2Code, DCGen, ReverseORC

Présentation à l'encadrant

August 11, 2026

Plan

1. Problématique
2. ScreenCoder
3. DesignCoder
4. Prototype2Code
5. DCGen
6. ReverseORC
7. Comparaison
8. Mise en perspective
9. Bibliographie

Problématique de recherche

Objectif

Transformer automatiquement une **maquette d'interface** en une **représentation structurée / hiérarchique**, servant d'étape intermédiaire avant la génération de code frontend.



Question centrale : comment chaque méthode passe-t-elle de la maquette à sa représentation structurée ?

Grille d'analyse utilisée

Pour chaque méthode étudiée, on cherche à identifier :

- les **données en entrée** (image seule ? métadonnées de design ? plusieurs tailles ?)
- la **façon d'analyser** la maquette (règles, MLLM, accessibilité API...)
- les **éléments détectés/extraits** et les informations associées
- la **détermination des relations** (parent-enfant, alignement, groupement)
- la **méthode de construction** du modèle (règles déterministes vs génératif)
- la **forme finale** du modèle (arbre ? DOM-like ? contraintes OR ?)
- les **étapes intermédiaires**
- l'**usage en aval** pour générer le code

ScreenCoder — vue d'ensemble

Jiang et al., *ScreenCoder: Advancing Visual-to-Code Generation for Front-End Automation via Modular Multimodal Agents*, 2025.

Idée clé : décomposer la tâche en 3 agents spécialisés pour corriger les erreurs de *perception* et de *planification* des MLLM monolithiques.



ScreenCoder — Grounding Agent

Entrée : image (screenshot ou croquis) uniquement.
Méthode : MLLM interrogé par des prompts type *"Where is the sidebar?"* $\Rightarrow$ ensemble

$$\mathcal{B} = \{(b_i, l_i) \mid l_i \in \mathcal{L}\}, \quad \mathcal{L} = \{\text{sidebar, header, navigation}, \ldots\}$$

Post-traitement (robustesse) :

- **Déduplication / NMS** par classe pour éliminer les détections redondantes
- **Fallback heuristique** si un composant clé manque (ex. boîte large et basse en haut $\rightarrow$ header)
- **Inférence du contenu principal** = plus grande zone rectangulaire non couverte

Sortie : un *dictionnaire de layout* (labels + boîtes) $\rightarrow$ fondation sémantique/spatiale.

ScreenCoder — Planning Agent (Algorithme 1)

Approche déterministe (pas générative) : *Visual-to-Structural Tree Mapping*.

Principe

1. Créer un nœud racine + un nœud par composant primaire (header, sidebar...)
2. Convertir coordonnées pixel → pourcentages responsives
3. Positionnement `absolute` pour préserver la macro-structure
4. Marquer les régions à enfants multiples comme conteneurs `CSS Grid`

Forme du modèle : arbre DOM-like, avec styles `position:absolute` + `grid` imbriqués. **Avantage revendiqué :** fidélité structurelle et prévisibilité (au prix de la flexibilité générative).

ScreenCoder — Génération et Placeholder Mapping

Generation Agent : parcourt l'arbre, génère HTML/CSS par nœud via prompt adaptatif (label sémantique + contexte structurel + instructions utilisateur optionnelles) $\rightarrow$ assemblage selon la hiérarchie.

Placeholder mapping (étape finale) :

- Détection des visuels réels (UIED) dans le screenshot source
- Partition par région sémantique (header, sidebar...)
- Appariement optimal *placeholders* $\leftrightarrow$ éléments réels (coût $= -\text{CIoU}$, résolu par **algorithme hongrois**)
- Insertion des images recadrées dans le code généré

Sert aussi de moteur de données : génère Screen-10K (10k paires image-code) pour fine-tuner un MLLM (SFT + RL avec récompense $-\text{MSE}$ pixel).

DesignCoder — vue d'ensemble

Chen et al., *DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with LLMs*, 2025.

Différence majeure : entrée = **maquette Figma/Sketch** (image + *métadonnées de design* : id, bbox, type, style) et non un simple screenshot brut.



DesignCoder — UI Grouping Chain (3 sous-tâches)

1. **Division (UI division)** : un LLM segmente la liste de couches L_{layers} (id, bbox, type) en sous-régions sémantiques D, avec correction algorithmique (Alg. 1) : fusion des régions qui se chevauchent, réassignation des couches orphelines à la région la plus proche, borne 3–10 régions.
2. **Extraction sémantique** : pour chaque sous-région, un MLLM décrit chaque élément (fonction, relations) à partir de l'image recadrée + métadonnées $\rightarrow L_{semantic}$.
3. **Groupement (Grouping)** : un prompt structuré organise $L_{semantic}$ en **arbre hiérarchique de composants**, avec post-traitement anti-chevauchement / anti-nœuds parasites.

Forme du modèle : arbre de composants *sémantiques* (pas seulement géométriques), chaque nœud portant une description fonctionnelle.

DesignCoder — Génération et auto-correction

Génération de code (divide-and-conquer) :

- Code composant : LLM "expert React" suit strictement l'arbre, assigne des identifiants de composants
- Style CSS : parcours de l'arbre *bottom-up* ; feuilles gardent leurs bbox, nœuds internes agrègent les bbox enfants (taille/position relative)

Self-correcting refinement (spécificité forte) :

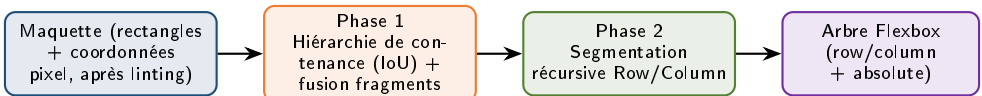
- Rendu réel (Appium) $\rightarrow$ comparaison image par image, composant par composant (*visual analysis prompt*)
- Génération de suggestions de correction $\rightarrow$ réparation ciblée et parallélisable par composant
- Boucle possible tant que des différences visuelles subsistent

Point clé : le modèle hiérarchique n'est pas figé après construction — il est *revisité* via une boucle de vérification visuelle.

Prototype2Code — vue d'ensemble

Xiao et al., *Prototype2code: End-to-end front-end code generation from UI design prototypes*, 2024 (méthode également utilisée comme baseline par ScreenCoder & DesignCoder).

Approche : purement à base de règles (pas de génération par LLM pour la structure) — construction **déterministe** d'un arbre "poupée russe" puis segmentation Flexbox.



Prototype2Code — Phase 1 : hiérarchie de contenance

1. Tri des groupes perceptuels (cartes, barres...) par taille **croissante**
2. Calcul d'**IoU** pour rattacher chaque couche à son groupe géométrique
3. **Fusion** des éléments fragmentés (ex. icône coupée) en une seule entité
4. Insertion des feuilles (triées par taille **décroissante**) sous le nœud le plus petit qui les contient → structure "poupée russe"
5. Si des enfants d'un nœud se chevauchent ⇒ nouveau parent avec `need_absolute=True` (positionnement CSS absolu)

Caractère : entièrement géométrique / à base de règles (pas de compréhension sémantique du contenu).

Prototype2Code — Phase 2 : segmentation Flexbox

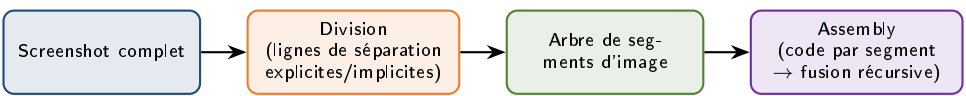
1. **Segmentation verticale** : recherche de coordonnées Y traversant la zone sans couper d'élément $\rightarrow$ "étages"
2. **Segmentation horizontale** : à l'intérieur de chaque étage, recherche de coordonnées $X \rightarrow$ colonnes
3. **Récursion** jusqu'à impossibilité de segmenter davantage
4. Attribution de `flex-direction: row` ou `column` selon l'alignement détecté

Forme finale : arbre où chaque nœud porte une instruction de mise en page explicite (Flexbox row/column, ou absolute) $\rightarrow$ mappage quasi direct vers HTML imbriqué + CSS.

DCGen (Divide-and-Conquer) — vue d'ensemble

Wan et al., *Divide-and-Conquer: Generating UI Code from Screenshots*, FSE 2025.

Constat (étude de motivation) : un MLLM appliqué directement produit 3 types d'erreurs — *omission*, *distorsion*, *mauvais agencement* d'éléments (85 % omission dans leur étude). Recadrer l'image en segments plus petits améliore nettement la qualité (similarité CLIP 73,7 %→76,0 %).



DCGen — Division : détection de lignes de séparation

Particularité : pas de détection d'éléments UI typés (pas de "sidebar/header"!) — on détecte des **lignes de coupure géométriques** dans l'image brute (niveaux de gris).

- **Lignes explicites** : bordures visibles (changement brutal de pixels)
- **Lignes implicites** : zones de faible variance (espaces blancs) encadrées par des transitions de luminosité (fenêtre glissante, double seuillage `var_thr`/`diff_thr`)
- Division **alterné et récursif** horizontal → vertical, jusqu'à profondeur max ou absence de nouvelle ligne

Forme du modèle intermédiaire : arbre de subdivisions *purement géométrique* (pas de sémantique de composants), chaque nœud = un segment d'image.

DCGen — Assembly : reconstruction du code

Deux variantes proposées :

Agent-based

- MLLM "leaf-solver" génère le code de chaque feuille
- MLLM "assembly" fusionne récursivement enfants → parent
- MLLM "generation" produit le code final (racine)

Rule-based

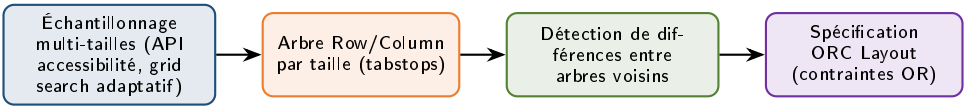
- Génération de code *uniquement* au niveau des feuilles
- Positionnement automatique via **CSS Grid** selon les positions enregistrées des segments
- Un LLM final ajuste la cohésion

Bilan : pas de modèle sémantique hiérarchique construit explicitement — la hiérarchie est *géométrique* (arbre de coupes), et sert de guide de *prompting* plutôt que de structure de sortie interprétable.

ReverseORC — vue d'ensemble

Jiang, Stuerzlinger, Lutteroth, *ReverseORC: Reverse Engineering of Resizable UI Layouts with OR-Constraints*, CHI 2021.

Particularité majeure : ce n'est pas un pipeline maquette-unique → code. L'entrée est un ensemble d'**échantillons du même UI à plusieurs tailles de fenêtre**, obtenus via API d'accessibilité (pas une image).



ReverseORC — Construction de l'arbre par taille

Extraction : API d'accessibilité → id, taille, position (top-left) de chaque widget (pas de reconnaissance d'image).
Tabstops : variables symboliques représentant les alignements (x-tabstop / y-tabstop). Un tabstop est un **layout divider** s'il coupe la maquette en deux sans traverser aucun widget.
Reconstruction récursive (façon XY-Cut) :

- Deux widgets → même *Row* s'ils sont entre les deux mêmes diviseurs horizontaux (idem *Column*)
- Subdivision horizontale prioritaire, puis verticale, sinon tabstops seuls (cas rare, ex. pinwheel)
- Regroupement de sous-couches consécutives si cela simplifie la structure

Forme : arbre Row/Column (comme Prototype2Code) mais recalculé **indépendamment à chaque taille échantillonnée**.

ReverseORC — Diff. d'arbres → patterns ORC

Détection de différences entre arbres de tailles voisines (édition d'arbres avec identités partielles) : `addNode`, `removeNode`, `moveNode`, `replaceNode`, `changeType`, `changeChildrenOrder`.
Mapping règle → pattern ORC (contrainte disjonctive "*A OU B*") :

- `removeNode` (feuille) → **widget optionnel**
- `moveNode` (enfants consécutifs) → fusion en **flux horizontal/vertical** ; sinon **position alternative**
- `replaceNode` → **layout alternatif**
- `changeType` → **sous-layout pivot** (Row ↔ Column)

Sortie finale : une **unique spécification ORC Layout**, valable sur *toute la plage de tailles*, avec une *error map* visualisant la fidélité de reconstruction (erreur structurelle, de transition, "fault lines").

Différences clés entre les méthodes

- **Règles vs apprentissage** : Prototype2Code, DCGen(division) et ReverseORC sont *déterministes / à base de règles* ; ScreenCoder (planning) est aussi déterministe mais le *grounding* utilise un MLLM ; DesignCoder utilise un LLM à *chaque* sous-étape.
- **Image seule vs métadonnées** : ScreenCoder & DCGen travaillent sur l'*image seule* ; DesignCoder & Prototype2Code exploitent des *métadonnées de design* (bbox/type/style) ; ReverseORC n'utilise *aucune image*, seulement les propriétés de widgets via API.
- **Détection d'abord vs hiérarchie directe** : ScreenCoder détecte des *régions labellisées* puis construit l'arbre ; Prototype2Code/ReverseORC/DCGen *ne labellisent pas* sémantiquement, ils coupent géométriquement l'espace.
- **Relations parent-enfant** : via *IoU* + inclusion (Prototype2Code), via *tabstops/layout dividers* (ReverseORC), via *lignes de séparation par variance* (DCGen), via *prompt de groupement sémantique* (DesignCoder), via *position absolue + grid* (ScreenCoder).
- **Nature du modèle final** : arbre *géométrique* pur (Prototype2Code, DCGen, ReverseORC) vs arbre *sémantique* enrichi (DesignCoder) vs arbre *DOM-like stylé* (ScreenCoder) vs *contraintes OR multi-tailles* (ReverseORC, unique dans ce corpus).

Similitudes entre les méthodes

- Toutes convergent vers une structure **arborescente récursive de type conteneur** (Row/Column, Grid, ou arbre DOM) : le *modèle hiérarchique* est un invariant conceptuel du domaine.
- Toutes (sauf ReverseORC) partent d'une **unique image/maquette statique**.
- Plusieurs méthodes (ScreenCoder, Prototype2Code, DCGen) séparent explicitement **détection/segmentation** et **génération de code**, illustrant un principe de *separation of concerns*.
- La **gestion des chevauchements** est un problème récurrent, résolu soit par positionnement absolu (Prototype2Code, ScreenCoder), soit par contraintes OR (ReverseORC).
- Plusieurs méthodes (ScreenCoder, DesignCoder, DCGen) exploitent un **découpage en sous-problèmes plus petits** pour pallier les limites de perception/planification des MLLM (*divide-and-conquer*).

Tableau comparatif global (1/2)

Critère	ScreenCoder	DesignCoder	Prototype2Code	DCGen	ReverseORC
Entrée	Screenshot / croquis (image seule)	Maquette Figma/Sketch + métadonnées	Maquette + métadonnées (post-linting)	Screenshot (image seule)	Widgets à N tailles (API accessibilité, pas d'image)
Analyse maquette	MLLM (grounding par prompts)	LLM (division sémantique)	Règles géométriques (IoU)	Détection de lignes (variance pixel)	Extraction API + tabstops
Détection composants	Régions labellisées (sidebar, header...)	Couches groupées sémantiquement	Groupes perceptuels fusionnés	Segments d'image non typés	Widgets individuels (typés par l'appli)

Tableau comparatif global (2/2)

Critère	ScreenCoder	DesignCoder	Prototype2Code	DCGen	ReverseORC
Relations	Position absolue + Grid	Groupement sémantique par LLM	IoU + inclusion géométrique	Lignes de séparation	Tabstops / layout dividers
Construction modèle	Déterministe (Alg. 1) sur sortie MLLM	LLM (prompt de groupement) + post-traitement	Règles (Alg. 1, poupée russe + Flexbox)	Récursion géométrique (XY-cut-like)	Récursion géométrique (XY-cut) $\times N$ tailles
Type de représentation	Arbre DOM-like stylé	Arbre de composants sémantiques	Arbre Row/Column Flexbox	Arbre de segments d'image	Arbre Row/Column $\rightarrow$ spéc. ORC (contraintes OR)
Génération code	Par nœud + placeholder mapping (Hongrois)	Divide-and-conquer + auto-correction visuelle	Mapping direct vers Flexbox	Assembly agent/rule-based	Solveur ORC (multi-plateforme)
Avantages	Robuste aux erreurs de perception	Sémantique riche, auto-corrigé	Simple, rapide, interprétable	Réduit omissions/distorsions	Seule à capturer le *redimensionnement*
Limites	Dépend du MLLM, de	Coût LLM élevé (plusieurs passes),	Pas de compréhension sé-	Pas de sémantique, de	Nécessite API, accessibilité pas

Constats pour ma problématique de recherche

- Le choix **image seule vs métadonnées** conditionne fortement la richesse du modèle : les métadonnées (DesignCoder, Prototype2Code) évitent les erreurs de perception mais supposent un accès au fichier de design (Figma/Sketch), non toujours disponible.
- Les approches **purement géométriques** (Prototype2Code, DCGen) donnent une structure *correcte mais sans étiquette sémantique* — utile pour la mise en page, insuffisant pour un code maintenable/accessible.
- Les approches **hybrides** (ScreenCoder : grounding sémantique + construction déterministe de l'arbre) semblent offrir le meilleur compromis robustesse/interprétabilité.
- **ReverseORC** montre qu'un modèle hiérarchique statique est insuffisant si l'objectif inclut le *comportement responsive* : il faudrait échantillonner ou raisonner sur plusieurs tailles, ou encoder des règles de flux (flow) directement dans le modèle.
- Les mécanismes d'**auto-correction visuelle** (DesignCoder) suggèrent qu'un modèle hiérarchique ne doit pas nécessairement être figé après une seule passe.

Caractéristiques souhaitées pour le modèle hiérarchique visé

1. **Structure arborescente** générique (conteneurs Row/Column/Grid imbriqués), indépendante du framework cible (HTML/CSS, React, Flutter).
2. **Étiquetage sémantique** des nœuds (rôle : header, navigation, card, liste...), pas seulement géométrique.
3. **Gestion explicite des cas ambigus** : chevauchement (positionnement absolu / z-index), éléments optionnels, alternatives de position.
4. **Robustesse au bruit de détection** (fallback heuristiques comme ScreenCoder, ou vérification visuelle comme DesignCoder).
5. **Possibilité (à terme) de capturer le redimensionnement** sans nécessiter un échantillonnage lourd multi-tailles (inspiration ReverseORC / OR-constraints, mais à partir d'une seule maquette si possible).
6. **Traçabilité vers la génération multi-cible** : le modèle doit rester assez générique pour piloter plusieurs générateurs de code (HTML/CSS, React, Flutter) sans re-analyse de la maquette.

Synthèse

À retenir

- Toutes les méthodes convergent vers une **structure arborescente de conteneurs**, mais diffèrent radicalement sur *comment* elles l'obtiennent (règles géométriques, MLLM, métadonnées, multi-échantillonnage).
- Le **compromis robustesse / richesse sémantique / coût** est le fil conducteur de la comparaison.
- ReverseORC introduit une dimension absente ailleurs : le **comportement dynamique** du layout.

Prochaine étape : définir le modèle hiérarchique cible pour la génération multi-technologies.

Bibliographie

- Jiang, Y., Zheng, Y., Wan, Y., Han, J., Wang, Q., Lyu, M. R., Yue, X. *ScreenCoder: Advancing Visual-to-Code Generation for Front-End Automation via Modular Multimodal Agents.* arXiv:2507.22827, 2025. `https://arxiv.org/abs/2507.22827`
- Chen, Y., Ding, S., Zhang, Y., Chen, W., Du, J., Sun, L., Chen, L. *DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with Large Language Models.* arXiv:2506.13663, 2025. `https://arxiv.org/abs/2506.13663`
- Xiao, S., Chen, Y., Li, J., Chen, L., Sun, L., Zhou, T. *Prototype2code: End-to-end front-end code generation from UI design prototypes.* IDETC/CIE 2024.
- Wan, Y., Wang, C., Dong, Y., Wang, W., Li, S., Huo, Y., Lyu, M. *Divide-and-Conquer: Generating UI Code from Screenshots.* Proc. ACM Softw. Eng. (FSE), 2025. `https://doi.org/10.1145/3729364`
- Jiang, Y., Stuerzlinger, W., Lutteroth, C. *ReverseORC: Reverse Engineering of Resizable User Interface Layouts with OR-Constraints.* CHI 2021. `https://doi.org/10.1145/3411764.3445043`

Merci — Questions ?